

mance. This type of learning approach is used in service robots to train them in certain environments and it doesn't require any training data sets or data models. Here, the Q-learning algorithm is used and works with the following formula:

$$Q_{\text{new}}(s_t, a_t) \leftarrow Q_{\text{old}}(s_t, a_t) + \alpha(r_t + \gamma \max_a Q(s_{t+1}, a) - Q_{\text{old}}(s_t, a_t))$$

Here,
 Q_{new} is the new state of the agent
 s_t is the state of the agent in time t
 a_t is the action taken by the agent in time t .

α is the learning rate
 r_t is the reward in time t
 γ is the discount factor

Example. If the agent takes actions (A1,A2,A3,A4), causes state changes (S1,S2,S3,S4), and receives a set of rewards R , then the Q values will be initialised and the rewards data will be created as follows:

Q =	States	A1	A2	A3	A4
	S1	0	0	0	0
	S2	0	0	0	0
	S3	0	0	0	0
	S4	0	0	0	0

R =	States	A1	A2	A3	A4
	S1	0		0	1
	S2	0	0		0
	S3		0	0	1
	S4	0		0	1

Here, the reward values are either 1 or 0 based on the actions.

If the agent is in state S4, then there are three possible actions that can change the state to S1, S3, and S4. This will be calculated using a simplified formula:

$$Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \gamma \times \text{Max}[Q(\text{next state}, \text{all actions})]$$

$$Q(S1, A4) = R(S1, A4) + 0.8 \times \text{Max}[Q(S4, A1), (S4, A3), (S4, A4)]$$

$$= 1 + 0.8 \times 0 = 1$$

Here, the value of $\text{Max}[Q(\text{next state}, \text{all actions})]$ is zero, because the Q values were initialised to zero.

So, as the new Q value for $Q(S1, A4)$ is 1, the Q values will be updated:

Q =	States	A1	A2	A3	A4
	S1	0	0	0	1
	S2	0	0	0	0
	S3	0	0	0	0
	S4	0	0	0	0

Similarly, the next states are determined by the agent, and the Q values will keep on updating.

The algorithm is:

1. Agent starts in state (s_t) and Q values are initialised.
2. Take action a_t and wait for a reward (r_t) and state (s_t) change.
3. Update the reward (r_t) and state (s_t) values.
4. Calculate the next action using the Q -learning formula.
5. Update Q values.

Among the three learning approaches, supervised learning is used only when both the input and output data is available along with the data model. Unsupervised learning is used only

when the input data is available, and reinforcement learning is required when there is no training data and the agent is required to be trained from the working environment. Apart from these, there are many other learning approaches like self-learning, feature learning, sparse dictionary learning, anomaly detection, robot learning, and association rule learning.

Hardware and software for ML

To run any computer program (algorithm) a central processing unit (CPU) is required. Most of the CPU hardware contains multiple cores, but such CPUs are designed for serial operations and do not provide high throughput. However, a graphics processing unit (GPU) can have a higher number of cores compared to a CPU and has higher throughput also. A CPU has more cache memory that can be used for complex operations, but a GPU can be used for simple operations even if it has lesser cache memory. That's why a GPU can be used with the help of compute unified device architecture (CUDA), which is also known as General Purpose computing on Graphics Processing Units (GPGPU).

Even though a GPU is useful for ML, it has certain limitations, one of which is that its architecture cannot be customised for specific purposes. To overcome such limitations, a special kind of chip (hardware) is used, called the field programmable gate array (FPGA). Such a chip can be pro-

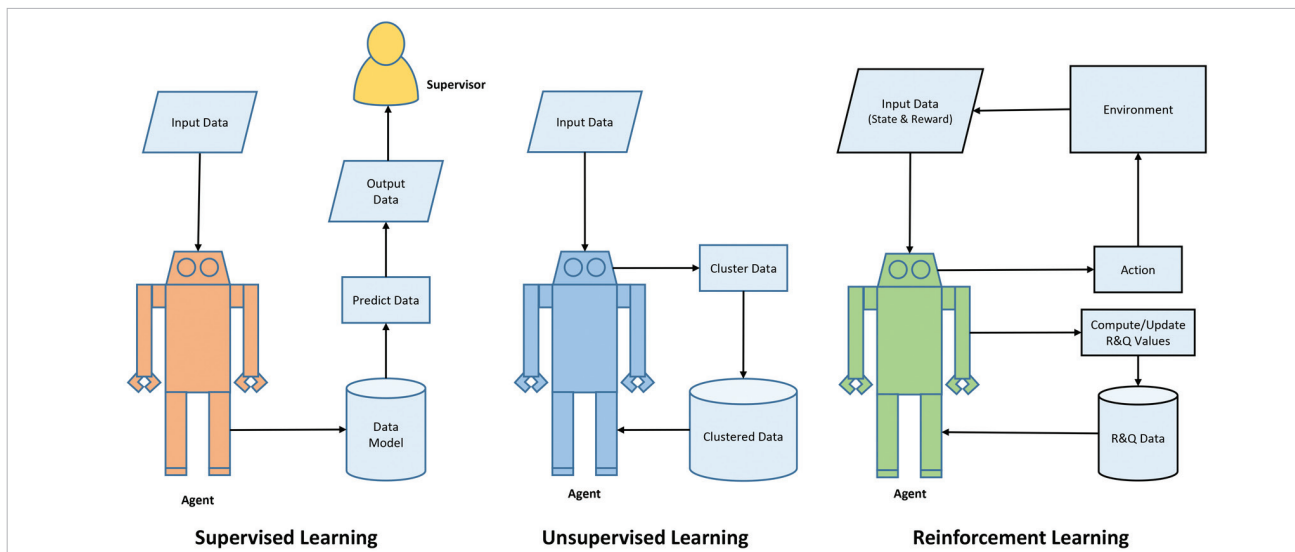


Fig. 1: Types of learning approaches